



Grado en Ingeniería Informática
Arquitectura de Computadores 25/26
Grupo 81

Proyecto de programación paralela
«Aplicación de síntesis de imágenes 3D»

Héctor Molina Garde — 100522253
Noa López Fernández — 100522230
Guillermo González Avilés — 100522146
Nicolás Maire Bravo — 100522100

Equipo 10

Profesor
José Daniel García Sánchez

Tabla de Contenidos

1. Modificaciones de diseño	2
1.1. Implementación de BVH.	2
1.2. Cambios en el randomizador.	2
1.3. Scripts de automatización de pruebas.	2
1.4. Diseño final del proyecto.	3
2. Evaluación rendimiento y energía	4
2.1. Proceso de paralelización	4
2.2. Motor de renderizado (rendering_engine.hpp)	4
2.3. Procesado de Imagen (image_par.cpp)	7
2.4. Solucion final.	8
3. Paralelización	9
3.1. Código original.	9
3.2. Funciones y algoritmos empleados	9
3.3. Cambios en el código.	10
3.4. Resultados	10
4. Organización de trabajo	11
4.1. División de trabajo	11
4.2. Desarrollo de tareas	13
5. Conclusiones	14
6. Uso de herramientas de IA	15
7. Bibliografía	16

1. Modificaciones de diseño

Pese a la solidez del diseño inicial, este presentaba algunas áreas a modificar de cara a la paralelización del código. Por ello, antes de comenzar la paralelización, se han realizado ciertas modificaciones de diseño.

1.1. Implementación de BVH.

Ya planteamos una implementación con *Axis Aligned Bounding Boxes* (AABB) en la primera práctica. Sin embargo, dada que la complejidad de los escenarios a evaluar ha aumentado, hemos decidido implementar una estructura *Bounding Volume Hierarchy* (BVH) que nos permite optimizar aún más las intersecciones de rayos y objetos. Su implementación ahora y no en la anterior entrega se debe al coste inicial de búsqueda de la intersección entre rayos y objetos, provocando que escenarios con pocos objetos demoren de un mayor tiempo en encontrarla (~10-20% más), pero logrando un mayor rendimiento en casos con gran cantidad de objetos como la última escena (~40-45% menos).

La estructura del BVH consiste en una jerarquía de volúmenes que agrupan los elementos de la escena, tratándolos como un árbol binario en el que cada caja ya definida en AABB se incluye en otra caja mayor que engloba varias de ellas, de este modo, podemos descartar el cálculos de intersección de grandes cantidades de objetos, pues si no hay intersección con la caja mayor, no es necesario comprobar las cajas interiores.

Además, se han realizado otras modificaciones relativas a la implementación de la estructura del BVH. Por ejemplo, la ampliación del código relativo a la AABB, añadiendo funciones que permiten la unión de dos AABB, calcular el centroide de una AABB y su eje más largo.

Por otro lado, tanto el renderizador como el parseador han sufrido modificaciones, aunque menores, para adaptarse a esta nueva estructura. En el *parser* de la escena, simplemente se ha añadido una llamada a la clase BVH, de modo que se genere de el árbol BVH con el que se trabajará para el renderizado. Por su parte, el renderizado ha sufrido modificaciones más agresivas, casi reescribiendo el funcionamiento estructural del módulo. Con la implementación del BVH se ha tenido que re implementar el renderizado bajo la detección de colisiones entre rayos y objetos.

De este modo, y con todas las modificaciones realizadas, se ha permitido una mejora sustancial en el rendimiento de la generación de imágenes, especialmente en aquellas con mayor complejidad.

1.2. Cambios en el randomizador.

En la primera práctica, dado que la ejecución era secuencial, el randomizador empleado no contaba con ningún tipo de seguridad para hilos. Por ello, y de cara a su uso en paralelo, evitando posibles secuencias o errores en la generación de imágenes, hemos tenido que reescribirlo.

En este caso, definimos las semillas de cada hilo como atómicas, haciendo que no se repitan entre hilos y no generen ningún tipo de secuencia o patrón. Además, para garantizar eficiencia y evitar la creación de más generadores aleatorios que hilos, hemos optado por definir los generadores como *enumerable_thread_specific*. De este modo, cada hilo cuenta con su propia instancia del generador aleatorio y evitamos posibles conflictos entre hilos.

1.3. Scripts de automatización de pruebas.

Pese a no formar parte explícita del código a desarrollar durante la práctica, nos parecía relevante hacer una breve mención a los scripts de automatización de pruebas realizados, ya que estos han sido un factor determinante en la eficacia y rapidez del desarrollo de la práctica en su totalidad.

Estos scripts permiten compilar, ejecutar casos de evaluación y descargar los “logs” e imágenes resultantes mediante *Makefile* para la simplificación de comandos en la terminal local y *sshpass* para automatizar la gestión del acceso a Avignon.

La compilación se realiza gracias a un script de *Bash* que despliega el código en Avignon y lanza una *SLURM* para la compilación. La ejecución se realiza mediante un *sbatch* en Avignon que ejecuta un *script* que se encarga de ejecutar con *perf* cada uno de los casos y guardar la salida del programa. Otros *scripts* permiten descargar los tiempos de ejecución y modificar los ficheros *.csv* con los que creamos las tablas de tiempo de versiones de la memoria.

1.4. Diseño final del proyecto.

Tras las modificaciones especificadas, tan solo se han producido cambios en las entidades y estructuras de datos del programa. La estructura y comportamiento de este sigue igual, más allá de los cambios de paralelización.

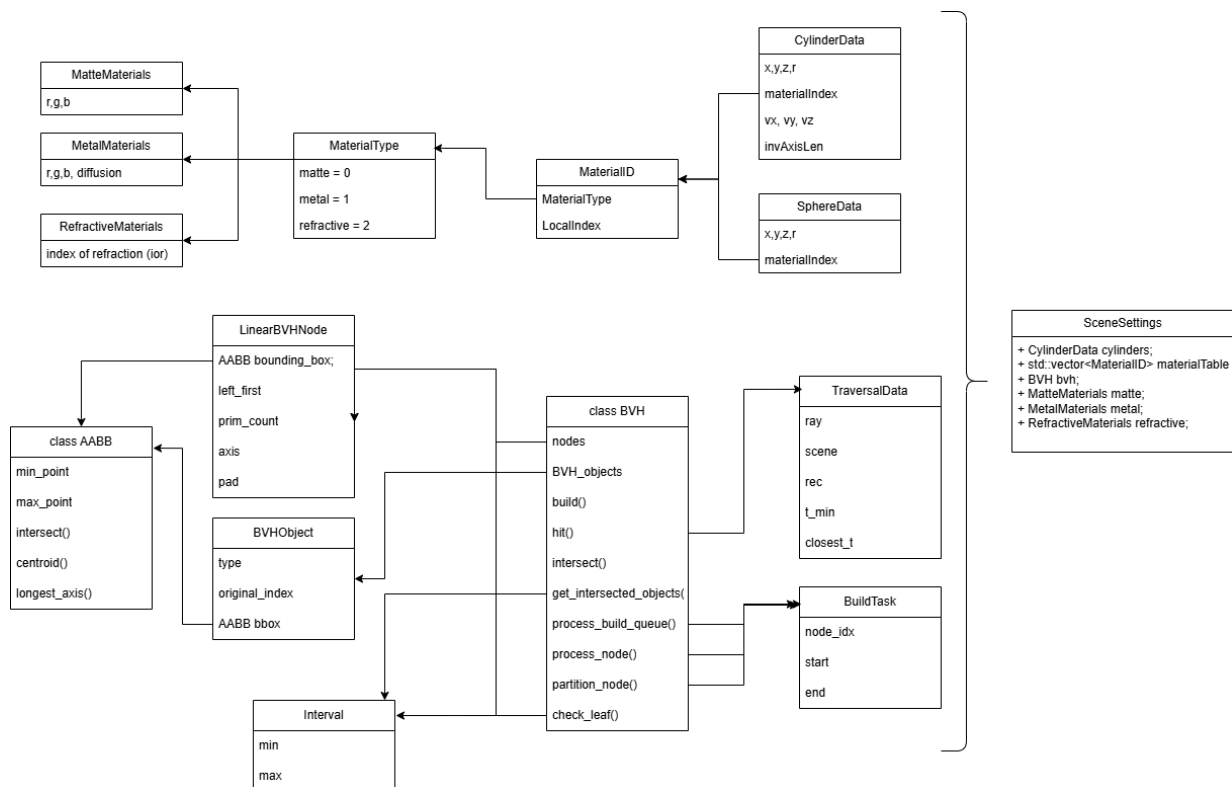


Figura 1: Esquema conceptual nuevo del proyecto

En la [Figura 1](#) se muestra el nuevo diseño del proyecto. Se ha incluido a *SceneSettings* el atributo *bvh*. Además se ha creado la clase *BVH* y modificado *AABBB*. El resto de estructuras que se muestran en el diagrama son envoltorios (*wrappers*) de datos para asegurar la baja aridad de las funciones, evitar la sobrecarga del código y realizar el principio de programación DIY (*Don't Repeat Yourself*).

2. Evaluación rendimiento y energía

En esta sección realizaremos un análisis del rendimiento y de la energía en base a diferentes factores, entre ellos: el número de hilos, granularidad y tipo de *partitioners*, permitiendo la comprensión de las decisiones de paralelización tomadas.

2.1. Proceso de paralelización

Para evaluar nuestro código y conseguir el mayor rendimiento y eficiencia energética, hemos realizado una evaluación exhaustiva de las mejores combinaciones de las métricas indicadas en la introducción de la sección para cada fragmento paralelizado (que se ha detallado en la sección anterior).

En primer lugar, hemos realizado un análisis del código inicial que nos ha permitido determinar que secciones merece la pena paralelizar. Hemos determinado que el fragmento del código que ocupa una mayor cantidad de tiempo con respecto a la ejecución completa del código es el modo de renderizado (*rendering_engine.hpp*), algo que dado que se trata de un proyecto de renderizado de imágenes tiene bastante sentido. Por otro lado, los dos siguientes fragmentos paralelizables que ocupan un mayor tiempo de ejecución (aunque significativamente menor) son *image_par.cpp* y *ppm_rwiter.cpp*.

Para evaluar estos fragmentos de manera independiente, se ha definido un proceso de evaluación fijo para todos los casos que se define a continuación:

- 1 En primer lugar establecemos el número de hilos del que empleará el fragmento. Para ello, antes de probar valores a fuerza bruta, comprobamos valores críticos en base a la arquitectura del nodo *stan*, como son: 1, 28, 56, 112 y 120 hilos. Esto, nos permitirá observar la tendencia de mejora en el rendimiento del código y poder elegir un rango sobre el que trabajar para nuestras pruebas.
- 2 Con el rango determinado y sin particionador, comprobamos todos los valores de hilos del rango más prometedor con un step 2. De este modo, podemos comprobar que valores presentan un mejor rendimiento y reducir aún más las pruebas que nos permitirán establecer el *partitioner* y el tamaño de grano.
- 3 Una vez hemos establecido un rango de valores reducido para el número de hilos (de unos 6-8 valores), probamos todas las combinaciones de granularidad y particionadores en cada uno de los valores, pudiendo así establecer cual es la combinación óptima y también identificar patrones al respecto.

2.2. Motor de renderizado (*rendering_engine.hpp*)

Dado que el motor de renderizado se presentaba como el mayor cuello de botella en nuestro código, su análisis resultaba especialmente relevante para la optimización del código.

Tras ejecutar la primera prueba mencionada con anterioridad, obtenemos los resultados presentados.

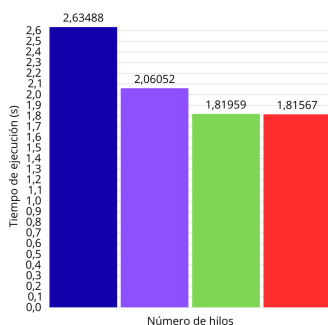


Figura 3: Evaluación rendimiento en base valores clave

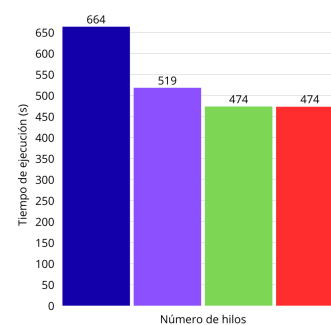


Figura 4: Evaluación consumo energético en base valores clave

Como se puede ver, en este caso (y en todos los que se presentarán al menos en esta sección de evaluación) el consumo de energía y el rendimiento del código están verdaderamente relacionados y presentan un comportamiento muy similar.

Más allá de eso, podemos ver como hay muy poca diferencia en la optimización entre las ejecuciones de 112 y 120 hilos, lo que nos puede dar a entender que el número óptimo de hilos debe estar cerca de 112 o ligeramente antes. Por tanto, procederemos a evaluar el rango de número de hilos de 56 a 120 hilos.

Por tanto, en las siguientes gráfica presentamos los resultados de rendimiento y energía de dicho rango, y podemos comprobar como los números de hilos que presentan mejores resultados se encuentran entre los 108 y los 114 hilos.

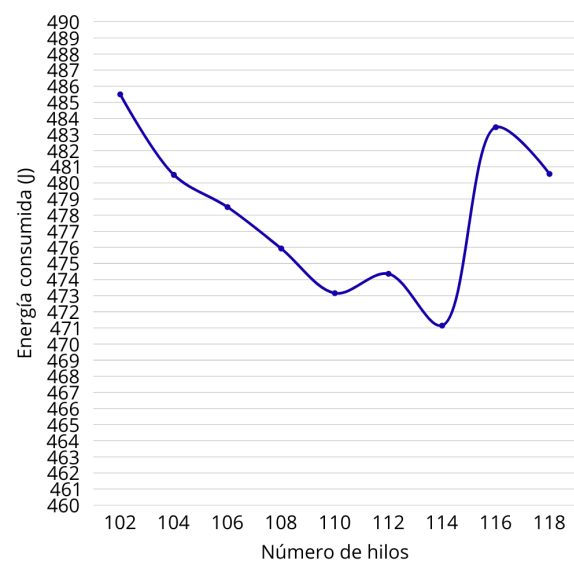
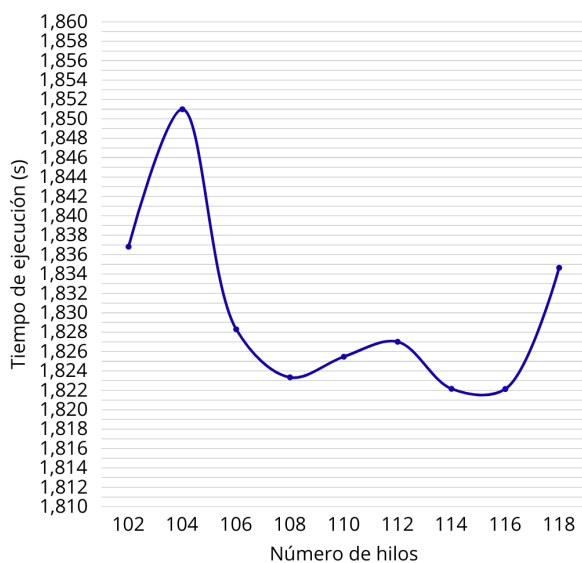


Figura 6: Evaluación rendimiento entre 108 y 114 hilos **Figura 7:** Evaluación consumo energético entre 108 y 114 hilos

Esto tiene total sentido, ya que son valores que se encuentran próximos al límite de paralelización físico que presenta la arquitectura de nuestra máquina.

Por ello, realizaremos nuestro estudio de particionadores y grano en base a la ejecución con 108, 110 y 112 hilos.

En este caso, hay cosas más significativas a observar y comentar que en las anteriores. Se puede ver en todos los casos, y para todos los valores posibles de granularidad, que el partitioner que presenta peor granularidad es el estático. Esto se puede atribuir a que el static_partitioner hace un reparto del trabajo de tamaño fijo y uniforme, siendo ideal para cargas de trabajo balanceadas y predecibles. Esto no encaja bien con el renderizado de píxeles, dado que hay una gran variación en el tratamiento que requieren en base a si en el se produce una intersección de rayos o no o del tipo de figura que se presenta. Y es precisamente esto por lo el simple_partitioner funciona especialmente bien para esta carga de trabajo, pues admite balanceo y adaptación para cargas de trabajo irregulares como es nuestro caso.

A continuación, se presentan las gráficas, que como se puede observar, presentan patrones muy similares tanto en rendimiento como en consumo energético.

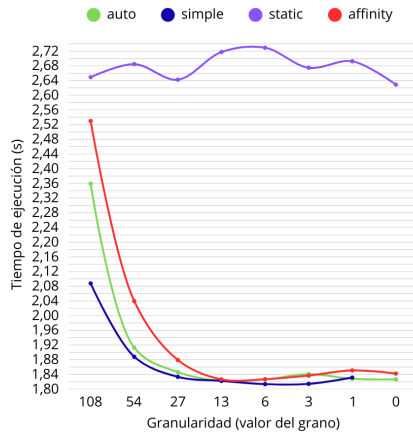


Figura 9: Evaluación rendimiento con 108 hilos

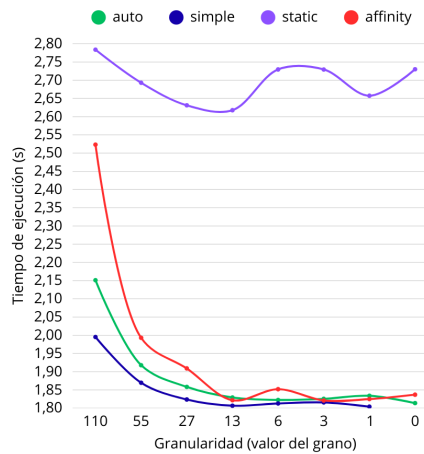


Figura 10: Evaluación rendimiento con 110 hilos

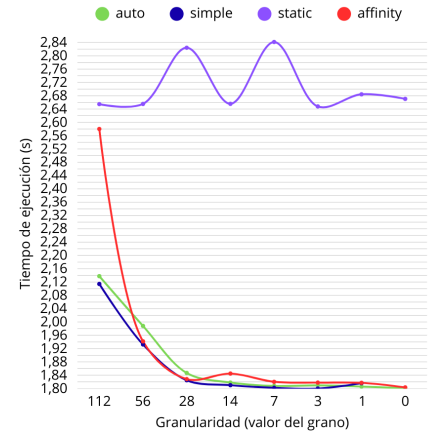


Figura 11: Evaluación rendimiento con 112 hilos

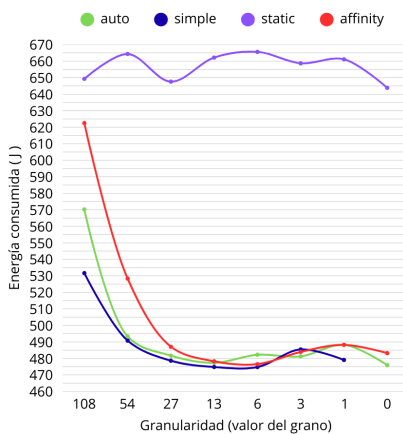


Figura 13: Evaluación rendimiento con 108 hilos

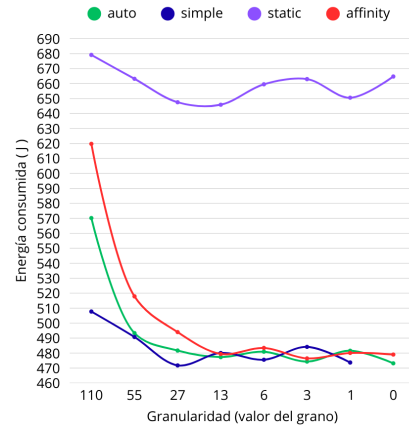


Figura 14: Evaluación rendimiento con 110 hilos

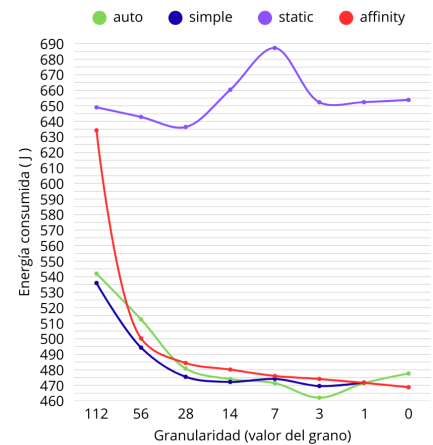


Figura 15: Evaluación rendimiento con 112 hilos

En cuanto a la granularidad, también presentan uniformidad, pues todos presentan valores óptimos con tamaño de grano 3. Esto es especialmente importante, y más tratándose de un bloque en 2 dimensiones. Con un grano 3 se permite el tratado de unos 9 píxeles por bloque, siendo estos lo suficientemente pequeños como para que el hilo esté balanceado de manera adecuada pero lo suficientemente grande como para amortizar el *overhead*. Además permite una buena localidad espacial.

Mirando a las gráficas, aunque se observa un comportamiento similar y unos valores cercanos, podemos observar que el valor óptimo se presenta con 112 hilos, tamaño de grano 3 y un particionador simple, con un tiempo de ejecución de 1,79998 segundos y un consumo energético de 469,5 J, lo que se traduce en un speedup de mayor a 22 respecto a la versión secuencial solo con la paralelización de este fragmento.

2.3. Procesado de Imagen (image_par.cpp)

Esta sección analiza el impacto de la paralelización en la etapa final de generación de imagen.

Para la evaluación de los efectos de la paralelización de la generación de la imagen, hemos seguido los mismos pasos seguidos en el caso anterior y comentados al comienzo de la sección.

No obstante, cabe destacar que, a diferencia del motor de renderizado, el procesado de imagen es una tarea con baja intensidad computacional pero alto volumen de accesos a memoria.

Tal y como se mencionó en la metodología a seguir, Se ejecutó un barrido de 1 a 120 hilos. El tiempo desciende de 38.36 s (1 hilo) a 37.73 s (120 hilos). El speedup se satura en 1.016x, validando la *Ley de Amdahl* que establece que dado $P_{seq} \approx 0.995$ (el renderizado secuencial consume 99.5%), el speedup teórico máximo es de $S_{max} = \frac{1}{P_{seq}} \approx 1.005x$.

Por otro lado, el consumo energético muestra tendencia ascendente: al no reducirse el tiempo, más hilos incrementan la potencia media resultando en penalización energética.

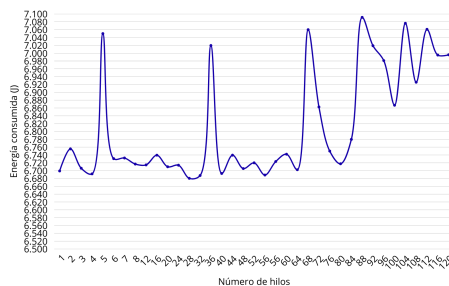


Figura 17: Consumo energético total vs hilos

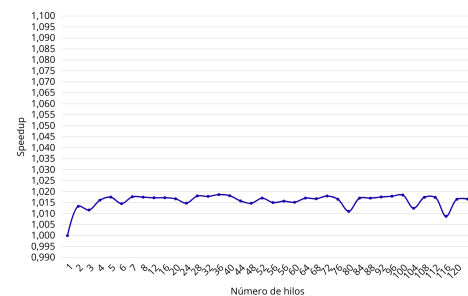


Figura 18: Curva de Speedup

Una vez establecido el número de hilos, determinamos la mejor combinación de *partitioners* y grano. Para ello, se ha llevado a cabo un barrido como el anterior comprobando las posibles combinaciones de estos para el número de hilos determinado.

Dados los resultados que se pueden observar en la gráfica de la derecha, el particionador que presenta mejores resultados es el *simple_partitioner*.

El comportamiento de este bajo diferentes granos muestra óptimo en: *simple_partitioner* con grano 64 (37.67s), esta combinación minimiza el compromiso entre balanceo de carga y *overhead* de gestión. No obstante, la mejora presentada con respecto a otras configuraciones es mínima, siendo de 7ms.

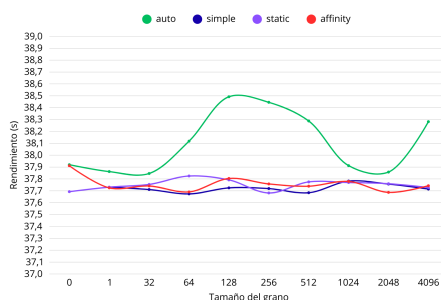


Figura 20: Comparativa de partitioners

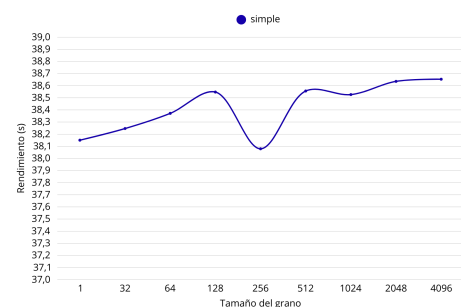


Figura 21: Detalle de granularidad para *simple_partitioner*

Una vez hemos realizado todos los pasos establecidos en nuestro estudio, podemos concluir que la paralelización es funcional y correcta. Basándonos en los datos empíricos, se selecciona la configuración simple/64 para la versión final, aunque se destaca que el sistema es insensible a variaciones en los parámetros de paralelización.

No obstante, como se puede ver, el speedup presentado en este caso es mínimo, muy a diferencia del caso que se ha presentado con anterioridad en el motor de renderizado. Por ello, no se aplicará la paralelización para este fragmento, y teniendo en cuenta que, como se ha mencionado al comienzo del documento, se trata del segundo fragmento del código que más tiempo ocupaba y que tenía posibilidad de paralelización, se ha decidido finalizar el estudio de paralelización y proceder con un modelo de paralelización en el que el único fragmento paralelizado es el motor de renderizador o *rendering_engine*.

2.4. Solucion final.

De este modo, la solución final planteada para el sistema pasa por la paralelización del bloque *rendering_engine* con los principios que se han establecido con anterioridad.

Dada esta configuración, observamos los siguientes datos para speedup y consumo energético final.

Tiempo de ejecución (s)	Energía consumida (J)	Speedup final
1.89	650.667	20.105

Estos, presentan unos resultado satisfactorios, que reducen significativamente el tiempo de ejecución con respecto al punto de partida inicial desde el que comenzó la paralelización. Por ello, podemos concluir que nuestra evaluación ha sido debidamente completada.

Además, pese a que no se presenta en esta práctica en concreto, planteamos la diferencia de duración para todas las imágenes del código de la primera práctica y el de esta ultima versión del código. Así como una variación del speedup para todas las imagenes, lo que nos permite evaluar debidamente los resultados obtenidos.

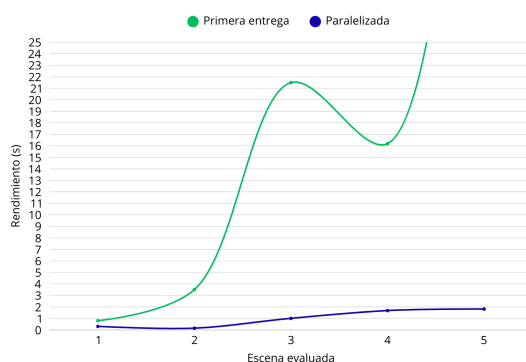


Figura 23: Comparativa de rendimiento en escenas con respecto a la primera entrega

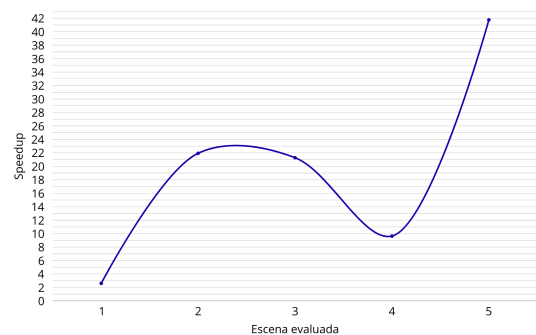


Figura 24: Incremento del speedup con respecto a las escenas estudiadas

Estas gráficas nos permiten establecer que los resultados han resultado en mejoras significativas en todas las imágenes, en especial en aquellas que tienen un mayor volumen de figuras, como es el caso de la quinta escena presentada para esta entrega, pues en ellas, los efectos de paralelización y de las estructuras BVH presentan un impacto mucho más notable. Además se presenta que en la versión paralelizada, el incremento en tiempo de ejecución y rendimiento según incrementa la complejidad de la imagen crece de manera mucho más lenta y controlada, lo que nos viene a demostrar la escalabilidad de nuestro programa y solución al problema de renderizado presentado en la práctica.

3. Paralelización

Tras la realización de la evaluación del apartado anterior, hemos conseguido una versión paralelizada del código de la primera entrega.

Para obtener los resultados mencionados a continuación se ha hecho uso de la librería TBB (Threading Building Blocks) que se presentó en clase como la herramienta a utilizar para la práctica.

3.1. Código original.

En nuestro caso, hemos paralelizado el fragmento del código que se encargaba del renderizado de las imágenes, que se encuentra más concretamente en el fichero *rendering_engine.hpp*.

El fichero mencionado se realiza el renderizado de la imagen. Para ello, se recibe la información de la escena tridimensional y se transforma la misma en la imagen en dos dimensiones que debemos representar. Así, recorreremos pixel a pixel la imagen y calculamos las intersecciones de rayos de los mismos que nos permiten generar las imágenes.

Este proceso, se lleva a cabo en un gran loop que recorre las dos dimensiones de la imagen (filas y columnas). Teniendo esto en cuenta, nos encontramos ante un bucle bidimensional con una carga de cálculo altísima para calcular los rayos (como ya vimos en la entrega anterior).

3.2. Funciones y algoritmos empleados

De este modo, hemos empleado diferentes recursos de la biblioteca.

En primer lugar, hemos utilizado un *parallel_for*. Este algoritmo, divide el bucle en fragmentos más pequeños que se ejecutan cada uno en un hilo. Para establecer el rango del bucle hemos utilizado un *blocked_range2d*. Hemos elegido este bloque precisamente inspirados por el trabajo de multiplicación de matrices de los laboratorios vistos en clase. En este caso, nos enfrentábamos a una información que puede ser comprendida como una matriz, en la que los elementos de la matriz son los píxeles sobre los que operamos, la anchura de la imagen el número de columnas y la altura el número de filas.

Además, hemos adaptado el algoritmo a los resultado empíricos que resultan en el enunciado anterior, con el que hemos podido establecer el número de hilos, el tamaño de grano y el tipo de particionador que funcionará en el bucle que hemos definido con anterioridad.

Para ello, primero de todo hemos establecido el límite de hilos que permitimos usar. En nuestro caso, según las pruebas realizadas el número óptimo de hilos es 112. Por ello, establecemos dicho límite con el uso de *global_control* que también pertenece a la librería TBB y que permite establecer el número de hilos que se usan en nuestra implementación paralela.

También establecemos el tamaño de grano. Esto, define el número de elementos que se le proporcionan a cada hilo. En nuestro caso, al usar un *blocked_range2d*, el tamaño de grano va a determinar bloques de $n \times n$ elementos (suponiendo un tamaño de grano n). Este valor, se añade como parámetro de la función *parallel_for* y, como se ha visto en el análisis del apartado anterior, toma un valor de 3, lo que indica que cada hilo recibe un bloque de 3×3 píxeles sobre el que operar. Este tamaño de grano, como ya se ha mencionado antes, permite un buen funcionamiento del programa ya que no es demasiado pequeño como para hacer que la operación consuma menos recursos de los que se gastan en emplear un hilo nuevo, pero a su vez es lo suficientemente pequeño como para que la carga de cómputo esté balanceada y no haya mucha diferencia en el tiempo de ejecución de los distintos hilos. En este mismo area, también hemos empleado un *simple_partitioner* que, como se ha visto en los laboratorios, permite la división hasta que llega al tamaño de grano asociado al rango. De este modo, el particionador refuerza el tamaño de grano y fomenta que se use la granularidad indicada y establecida. Este tipo de particionador funciona especialmente bien para este proceso concreto ya que cada pixel requiere una cantidad de trabajo muy similar, por lo que las cargas

de trabajo están balanceadas y por tanto no necesitamos un *partitioner* que gestione las cargas de manera «automática».

3.3. Cambios en el código.

Empleando lo que se ha mencionado, el bucle de renderizado pasa de ser:

```
for (size_t row = 0; row < imageHeight; row++) {
    for (size_t col = 0; col < imageWidth; col++) {

        // LÓGICA DE INTERSECCIÓN DE RAYOS
    }
}
```

A una lógica más compleja con el uso de la librería TBB, de modo que se emplean todas las funciones y herramientas que se han mencionado con anterioridad:

```
tbb::global_control global_limit(tbb::global_control::max_allowed_parallelism, 112);

tbb::parallel_for(
    tbb::blocked_range2d<size_t>(0, imageHeight, 3, 0, imageWidth, 3),
    [&](tbb::blocked_range2d<size_t> const & r) {

        // GENERADORES LOCALES

        for (size_t row = r.rows().begin(); row != r.rows().end(); ++row) {
            for (size_t col = r.cols().begin(); col != r.cols().end(); ++col) {

                // MISMA LÓGICA DE INTERSECCIÓN DE RAYOS
            }
        }
    },

    tbb::simple_partitioner()
);
```

Como se puede ver, la segunda implementación, que se corresponde con el código paralelizado, es más compleja y engorrosa, pues contiene todas las limitaciones que se han mencionado con anterioridad.

3.4. Resultados

Dado que, como ya se ha mencionado en el apartado anterior de la memoria, hemos establecido que esta paralelización es suficiente para obtener buenos resultados, pues la paralelización de otros fragmentos supondría una mejora prácticamente insignificantes del rendimiento y el consumo de la energía.

Por ello, nuestro análisis de paralelización concluye con el renderizado de las imágenes, pues no hay más fragmentos del código que hayan requerido de su paralelización (pues las modificaciones en el generador de números aleatorios no se incluyen en este apartado si no en el de modificaciones presentado con anterioridad).

4. Organización de trabajo

Dado que en la entrega previa del proyecto la organización del trabajo fue verdaderamente eficaz y nos permitió entregar el trabajo a tiempo y con unos buenos resultados, hemos decidido utilizar de nuevo el método de organización por sprints para la realización de esta segunda parte, empleando herramientas de desarrollo Agile como Jira, que nos permiten saber en todo momento las tareas pendientes, pudiendo así, tener una visión completa del estado y evolución del trabajo.

En este caso, dado que tanto la cantidad de código a producir como el tiempo de trabajo ha sido significativamente menor, los sprints son de menor duración y cantidad de tareas, no obstante, se han mantenido cuatro fases de trabajo como en la entrega previa y se han incluido a la planificación las tareas esenciales a realizar en cada sprint desde el primer momento.

En la siguiente imagen, podemos comprobar cual ha sido la planificación inicial de sprints en un calendario, representando cada color una etapa diferente de trabajo en el calendario desde la publicación del proyecto hasta la entrega del mismo.

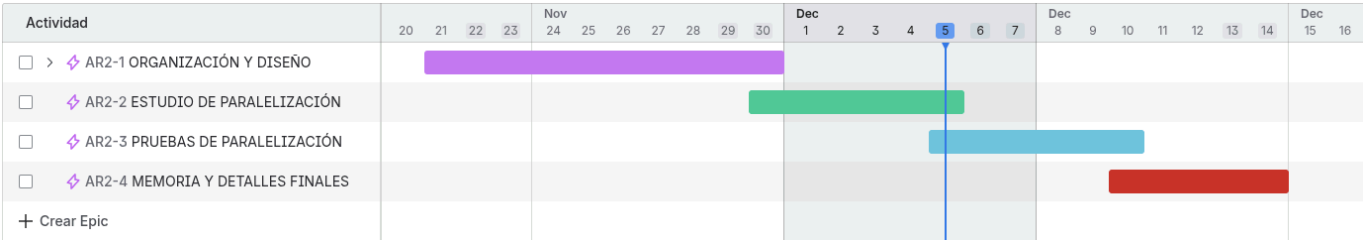


Figura 25: Calendario de sprints para la entrega

4.1. División de trabajo

La subdivisión en secciones de trabajo y sus correspondientes sprints se incluyen a continuación:

1. **Organización y diseño:** En esta etapa, se han realizado la creación y modificación del repositorio de modo que se pueda paralelizar adecuadamente. Esto incluye la modificación del randomizador y modificaciones como la inclusión del modelo BVH.

Esta etapa es realmente importante dado que nos permite establecer una base sólida para el proyecto. Por ello, como se puede ver en el calendario de sprints presentado, su duración es mayor, pues un buen trabajo en el mismo nos permitirá encontrar menos contratiempos y errores durante el resto de la práctica.

Actividad	Persona asignada	Informador	Estado	Fecha de vencimiento	Resolución
> AR2-4 MEMORIA Y DETALLES FINALES	Sin asignar	Noa López Fern...	TAREAS POR HACER	14 dic 2025	Sin resolver
> AR2-3 PRUEBAS DE PARALELIZACIÓN	Sin asignar	Noa López Fern...	TAREAS POR HACER	10 dic 2025	Sin resolver
> AR2-2 ESTUDIO DE PARALELIZACIÓN	Sin asignar	Noa López Fern...	TAREAS POR HACER	05 dic 2025	Sin resolver
∨ AR2-1 ORGANIZACIÓN Y DISEÑO	Sin asignar	Noa López Fern...	TAREAS POR HACER	30 nov 2025	Sin resolver
☑ AR2-9 Creación nuevos scripts de trabajo	Héctor Molina	Noa López Fern...	FINALIZADA	30 nov 2025	Listo
☑ AR2-8 Implementación BVH	Noa López Fernández	Noa López Fern...	FINALIZADA	30 nov 2025	Listo
☑ AR2-7 Randomizador paralelizable	Noa López Fernández	Noa López Fern...	FINALIZADA	30 nov 2025	Listo
☑ AR2-6 Ajustes repo para tbb	Héctor Molina	Noa López Fern...	FINALIZADA	30 nov 2025	Listo
☑ AR2-5 Creación de nuevo repo	Héctor Molina	Noa López Fern...	FINALIZADA	30 nov 2025	Listo

Figura 26: Sprint 1 de tareas: Organización y diseño.

2. **Estudio de paralelización:** Una vez sabemos cual es nuestro punto de partida, hemos dedicado cierto tiempo a establecer y estudiar de manera más teórica posibilidades de paralelización e incluso comprobando algunas de ellas.

Esto nos permite establecer los cuellos de botella de nuestro código y así poder enfocar nuestras energías en fragmentos concretos del código que nos van a ofrecer una mayor mejora en el código.

Actividad	Persona asignada	Informador	Estado	Fecha de vencimiento	Resolución
> AR2-4 MEMORIA Y DETALLES FINALES	Sin asignar	Noa López Fern...	TAREAS POR HACER	14 dic 2025	Sin resolver
> AR2-3 PRUEBAS DE PARALELIZACIÓN	Sin asignar	Noa López Fern...	TAREAS POR HACER	10 dic 2025	Sin resolver
✓ AR2-2 ESTUDIO DE PARALELIZACIÓN	Sin asignar	Noa López Fern...	TAREAS POR HACER	05 dic 2025	Sin resolver
✓ AR2-15 Decisión de secciones a paralelizar	Noa López Fernández	Noa López Fern...	FINALIZADA	Ninguno	Listo
✓ AR2-14 Prueba de cambios en modelos de paraleliz...	Sin asignar	Noa López Fern...	FINALIZADA	Ninguno	Listo
✓ AR2-13 Scripts de prueba	Héctor Molina	Noa López Fern...	FINALIZADA	Ninguno	Listo
✓ AR2-12 Pruebas de paralelización y funcionamiento...	100522100	Noa López Fern...	FINALIZADA	Ninguno	Listo
✓ AR2-11 Búsqueda de cuellos de botella y revisión de...	Noa López Fernández	Noa López Fern...	FINALIZADA	Ninguno	Listo
✓ AR2-10 Análisis de secciones a paralelizar	Gui	Noa López Fern...	FINALIZADA	Ninguno	Listo

Figura 27: Sprint 2 de tareas: Estudio de paralelización

3. **Búsqueda de solución óptima:** Una vez hemos determinado los cuellos de botella de nuestro código en el sprint anterior, utilizamos esta etapa para realizar pruebas de granularidad, numero de hilos etc. de modo que podamos exprimir al máximo la eficiencia de los fragmentos paralelizados, consiguiendo los mejores tiempos posibles.

Actividad	Persona asignada	Informador	Estado	Fecha de vencimiento	Resolución
> AR2-4 MEMORIA Y DETALLES FINALES	Sin asignar	Noa López Fern...	TAREAS POR HACER	14 dic 2025	Sin resolver
✓ AR2-3 PRUEBAS DE PARALELIZACIÓN	Sin asignar	Noa López Fern...	TAREAS POR HACER	10 dic 2025	Sin resolver
✓ AR2-20 Scripts para ejecución automática	100522100	Noa López Fern...	EN CURSO	Ninguno	Sin resolver
✓ AR2-19 Estudio punto de partida secuencial	Héctor Molina	Noa López Fern...	EN CURSO	Ninguno	Sin resolver
✓ AR2-18 Estudio paralelización writer_ppm	Gui	Noa López Fern...	EN CURSO	Ninguno	Sin resolver
✓ AR2-17 Estudio paralelización image_par	100522100	Noa López Fern...	EN CURSO	Ninguno	Sin resolver
✓ AR2-16 Estudio de paralelización de rendering engine	Noa López Fernández	Noa López Fern...	EN CURSO	Ninguno	Sin resolver

Figura 28: Sprint 3 de tareas: Búsqueda de solución óptima.

4. **Memoria y últimos detalles:** Para finalizar, una vez tenemos las mejores combinaciones de paralelización de cada fragmento paralelizado, juntamos dichas combinaciones establecemos unos tiempos de ejecución finales y pasamos toda la documentación correspondiente a la memoria para su entrega.

Actividad	Persona asignada	Informador	Estado	Fecha de vencimiento	Resolución
✓ AR2-4 MEMORIA Y DETALLES FINALES	Sin asignar	Noa López Fern...	TAREAS POR HACER	14 dic 2025	Sin resolver
✓ AR2-23 Poda de ramas innecesarias y merges nece...	Sin asignar	Noa López Fern...	TAREAS POR HACER	Ninguno	Sin resolver
✓ AR2-22 Preparación del repositorio final	Sin asignar	Noa López Fern...	TAREAS POR HACER	Ninguno	Sin resolver
✓ AR2-21 Finalizar memoria (colectivo)	Sin asignar	Noa López Fern...	TAREAS POR HACER	Ninguno	Sin resolver

Figura 29: Sprint 4 de tareas: Memoria y últimos detalles.

4.2. Desarrollo de tareas

Es importante destacar que las tareas son orientativas y se preparan al comienzo del trabajo y por tanto pueden sufrir modificaciones en el momento de su realización o es posible que se realicen tareas que ocurran de imprevisto y no estén incluidas en dichos sprints.

Por ello, además de los sprints presentados en la sección anterior que presentan las principales tareas a realizar de manera general, la subdivisión en subtareas dentro de dichos sprints, hemos generado una hoja de cálculo en la que se especifican detalladamente la realización de las diferentes subtareas por los diferentes miembros del grupo agrupadas por sprint, así como las horas dedicadas a cada una y la fecha de realización. Dicha hoja de cálculo se puede encontrar en el siguiente link: [Organización de trabajo del equipo 10](#)

5. Conclusiones

En definitiva, consideramos que los resultados de la práctica han sido muy satisfactorios, presentando un speedup superior a 40 en la escena 5 y con un tiempo de ejecución aproximado de 1,8 segundos frente a los 175 que se presentan como valor máximo de referencia, lo que supone un tiempo casi 100 veces menor.

Más allá de los resultados obtenidos, consideramos que la práctica ha sido muy útil para nuestra formación en el contexto de la programación paralela, y más específicamente en el empleo de la librería TBB. En nuestro caso, nunca habíamos realizado programación paralela y poder aplicar los conocimientos adquiridos a un proyecto práctico, nos ha resultado muy interesante y útil.

La práctica además nos ha ayudado a desarrollar mejores cualidades de análisis de rendimiento, comparativa de ejecuciones y organización de pruebas. En comparación con la primera entrega, el volumen de trabajo de evaluación y pruebas ha sido notoriamente superior, por lo que hemos tenido que desarrollar una estrategia más sofisticada y organizada de lo que lo hicimos en la primera entrega.

En cuanto a aspectos a mejorar para futuros proyectos, destacamos el empleo de una mejor organización. En este caso, y pese a seguir una organización similar a la de la primera práctica, el tiempo se nos ha echado más encima. Nos habría gustado poder realizar alguna prueba más de evaluación para contrastar los resultados obtenidos, no obstante, dado que el tiempo de ejecución del trabajo era muy limitado y que esto ha conllevado la saturación del sistema *Avignon*, nos ha resultado algo complicado la realización de las mismas.

En este caso, dado que la carga de programación era notoriamente inferior a la de la primera práctica, nuestras dificultades en ese ámbito han sido prácticamente inexistentes. EL único aspecto que nos ha dificultado ligeramente el trabajo (a parte de las esperas para ejecución y pruebas) ha sido la evaluación de los fragmentos de manera independiente. Dado que analizábamos cada fragmento de manera independiente, al combinar todos los resultados obtenidos y las combinaciones óptimas, sufrimos de problemas de *merge* para las ramas que habíamos generado para la evaluación. Por ello, si tuviéramos que empezar de nuevo el proyecto, organizaríamos el trabajo de evaluación de una manera más secuencial en la que pudiéramos juntar los resultado obtenidos sin tantos problemas y conflictos.

No obstante, estamos satisfechos con el trabajo realizado, con la organización como equipo de trabajo y consideramos que la práctica ha sido interesante y nos ha obtenido beneficios notorios no solo desde los conocimientos de la asignatura, también como programadores y estudiantes de ingeniería informática.

6. Uso de herramientas de IA

En el desarrollo de este proyecto, y conforme a las directrices establecidas en el enunciado y en los estándares establecidos en el departamento, tal y como ya se hizo en la primera entrega de la práctica, a lo largo de la realización del proyecto, hemos empleado diversas herramientas de Inteligencia Artificial como apoyo para la productividad, la organización y sobre todo, la resolución de errores y bugs en el código desarrollado.

A continuación, se detalla el rol específico que cada modelo ha desempeñado en nuestro flujo de trabajo, destacando, como es evidente, que la responsabilidad final sobre el diseño, implementación y verificación del código recae exclusivamente sobre nosotros, los integrantes del grupo.

Pese a que el uso de dichas herramientas ha variado en base al integrante del grupo, en términos generales, se han empleado las siguientes herramientas, que se mantienen muy similares a las empleadas en la entrega anterior del proyecto:

1. Gemini 3 Pro: Asistente de Desarrollo Integral:

Gemini 2.5 Pro fue la herramienta más utilizada a lo largo del proyecto, principalmente por su actualización al comienzo del mismo, cosa que ha incrementado notablemente su utilidad. Podemos dividir su uso en las siguientes áreas principales:

- Validación de organización en la estructura del proyecto y sugerencias de mejora.
- Ayuda de diseño y *brainstorming* de desarrollo del proyecto.
- Comprobaciones de código rápido y depurado de errores sencillos.

En diferencia a la entrega anterior, y pese a seguir sin ser la herramienta más útil para la resolución de errores complejos, la actualización ha favorecido mucho su uso a lo largo de la realización del proyecto. Además, dada su velocidad de respuesta y agilidad resulta una herramienta muy útil para un proyecto de este tipo, en el que no hay una cantidad abundante de código pero si de comprobaciones y modificaciones, que pueden dar lugar a pequeños errores de fácil resolución, que Gemini 2.5 Pro es capaz de solventar de manera rápida y eficaz frente a otros LLM con un tiempo de respuesta y «razonamiento» mucho más lento.

2. Deepseek: Especialista en Problemas Complejos:

Recurrimos a Deepseek de manera notoriamente más selectiva, principalmente cuando nos enfrentábamos a complejos *bugs* que resultaban incapaces de solucionar con otros LLM. Su uso viene condicionado principalmente por la velocidad de respuesta de dicha herramienta. Su lentitud de razonamiento lo hace muy eficaz para la resolución de problemas más complejos y el depurado de código, no obstante, la pérdida de agilidad que supone limita su uso a casos aislados.

Es más, en esta entrega, dado que la cantidad de código a producir ha sido significativamente menor que en el caso de la entrega anterior, y por tanto el número de bugs críticos también se ha visto reducido, el uso de Deepseek ha sido mucho más limitado, centrándose casi exclusivamente en la resolución de problemas relacionados con la implementación en BVH que no fuimos capaces de resolver con Gemini.

En conclusión, el uso de herramientas de IA ha sido un apoyo valioso en la realización de este proyecto, facilitando tanto la organización como la velocidad de realización de la misma.

7. Bibliografía

- [1] Intel, *Threading Building Blocks Developer Guide and API Reference*, [Online] URL: <https://www.intel.com/content/www/us/en/docs/onetbb/developer-guide-api-reference/2021-9/parallel-for.html>
- [2] Roman Wiche, *Another View on the Classic Ray-AABB Intersection Algorithm for BVH Traversal*, Medium 29 Septiembre 2018, [Online] URL: <https://medium.com/@bromanz/another-view-on-the-classic-ray-aabb-intersection-algorithm-for-bvh-traversal-41125138b525>
- [3] Wikipedia, *Bounding volume hierarchy*, 28 Octubre 2025 [Online] URL: https://en.wikipedia.org/wiki/Bounding_volume_hierarchy
- [4] Jbikker, *How to build a BVH – Part 1: Basics*, Ompf2 13 Abril 2022, [Online] URL: <https://jacco.ompf2.com/2022/04/13/how-to-build-a-bvh-part-1-basics/>
- [5] Intel, *Threading Building Bocks Deveoper Guide and API Reference*, [Online] URL: <https://www.intel.com/content/www/us/en/docs/onetbb/developer-guide-api-reference/2021-9/partitioner-summary.html>